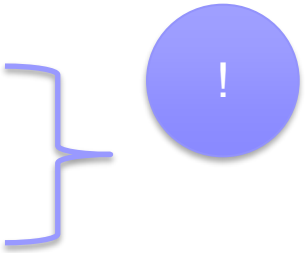
Blerina Spahiu / Matteo Palmonari

RDF & SPARQL: Exercises

Practical Recap + Examples

Overview

- Recap on RDF
- Recap on SPARQL



Indicates content that is more interesting from a pragmatic point of view and/or novel w.r.t. theory slides

This presentation

- Exercises in RDF + SPARQL
  - For RDF exercise this year also consider
    - DS2526 - ESE1.0 - Exercises about RDF modeling
- Exercises in Advanced SPARQL

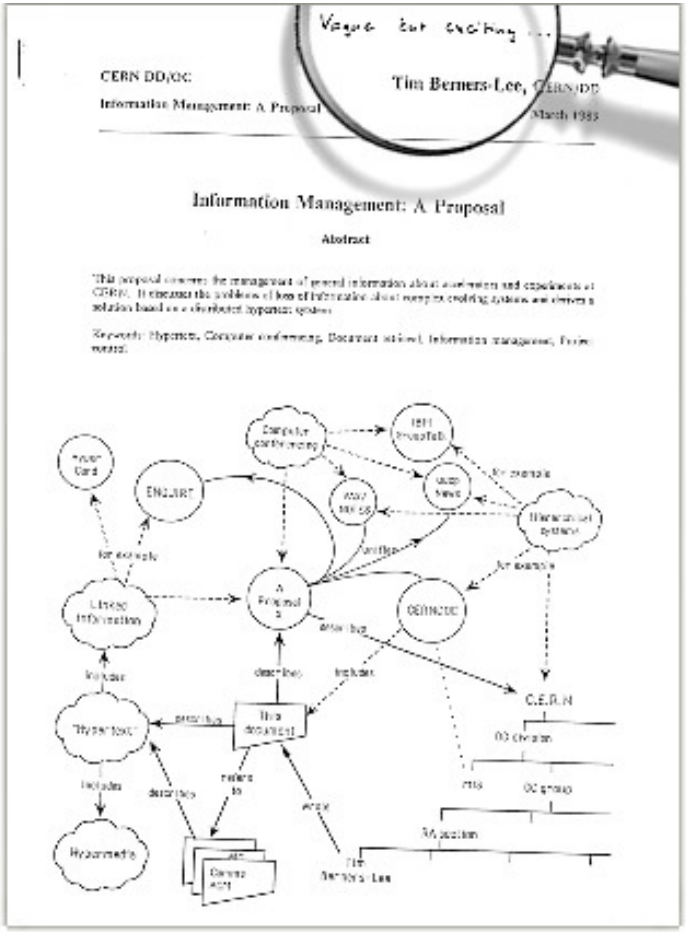
1. RDF Recap

Recap and more about syntax

Describe Resources on the WEB

Tim's proposal[1]:

A graph representation of the knowledge



1. http://info.cern.ch/Proposal.html

RDF Model

George Clooney birthplace is Lexington Us and he was born on 6 May 1961

(Subject, predicate, object)

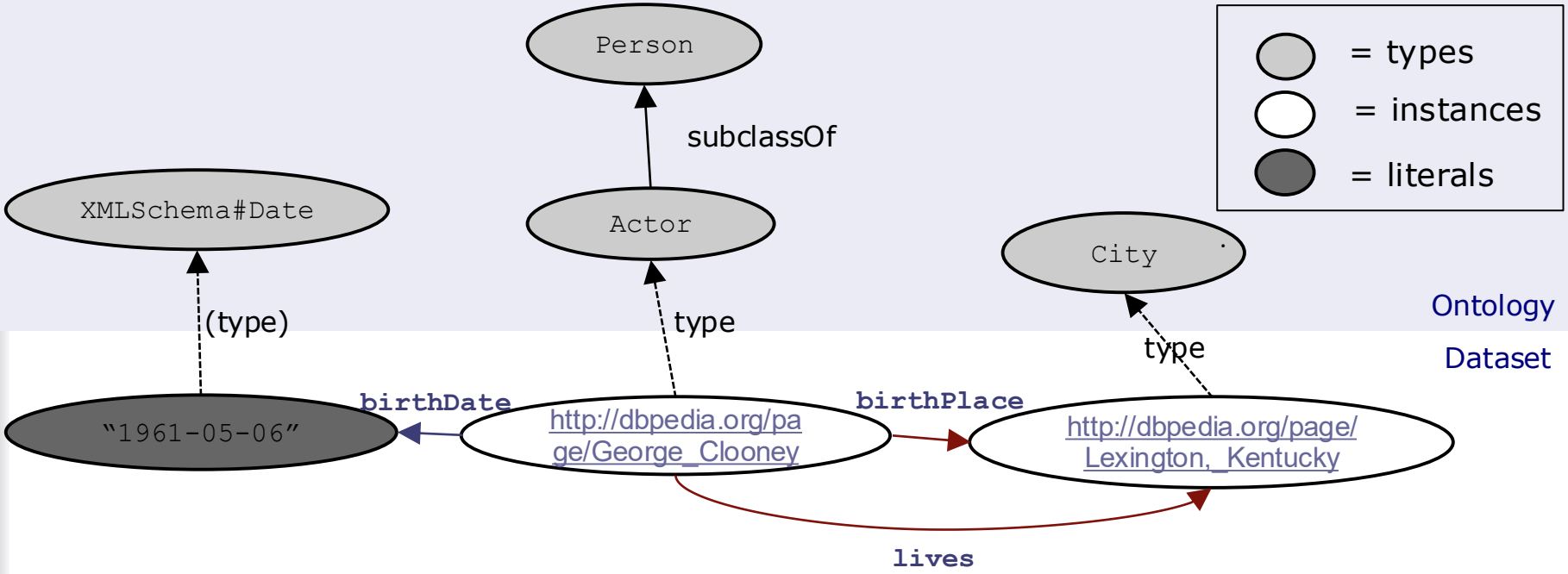
(George Clooney, birthplace, Lexington US)

(George Clooney, birthdate, 6 May 1961)

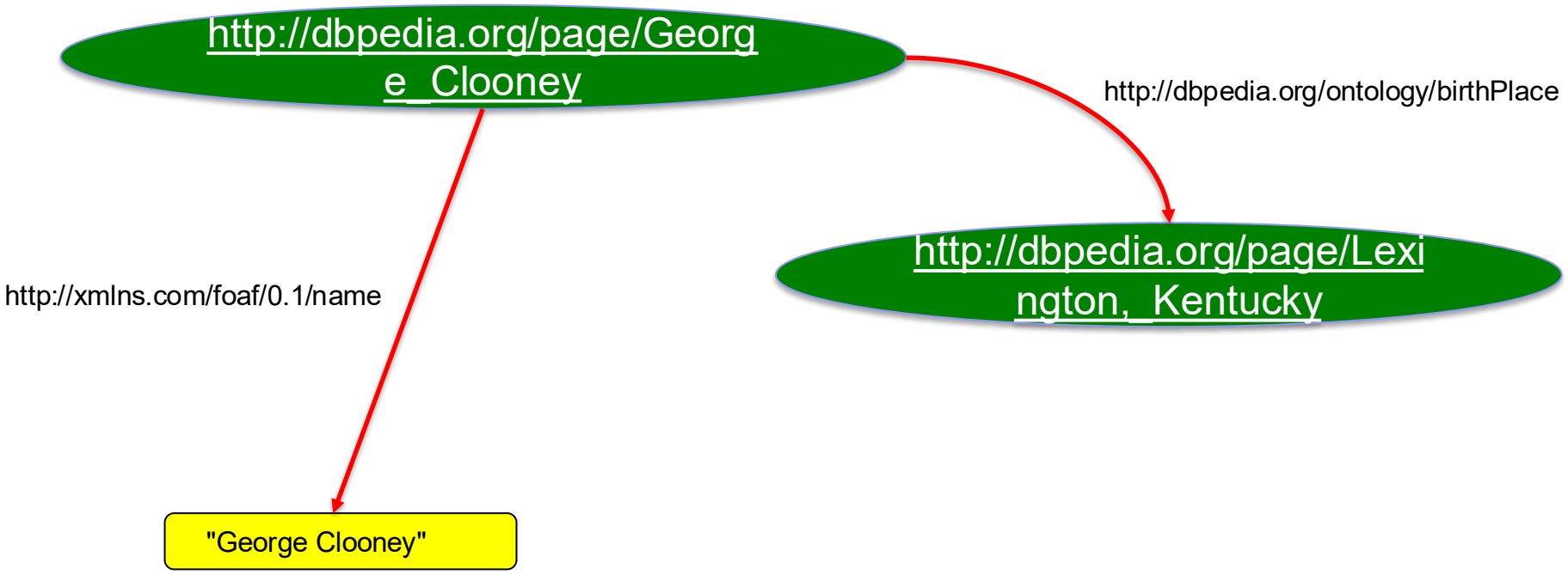
The subject is a resource (not a literal)
The predicate is identified by a URI
The object is either a resource or a literal

RDF oriented labeled multigraph model

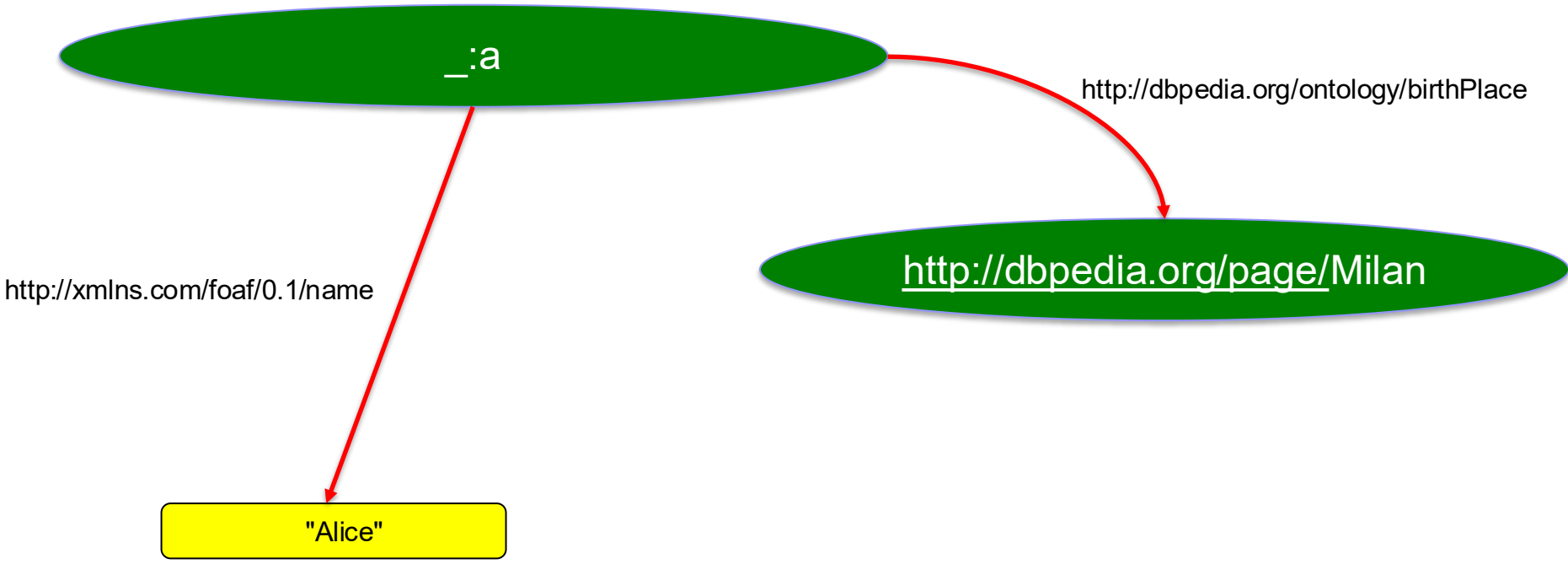
George Clooney birthplace is Lexington Us and he was born on 6 May 1961



RDF Conventions

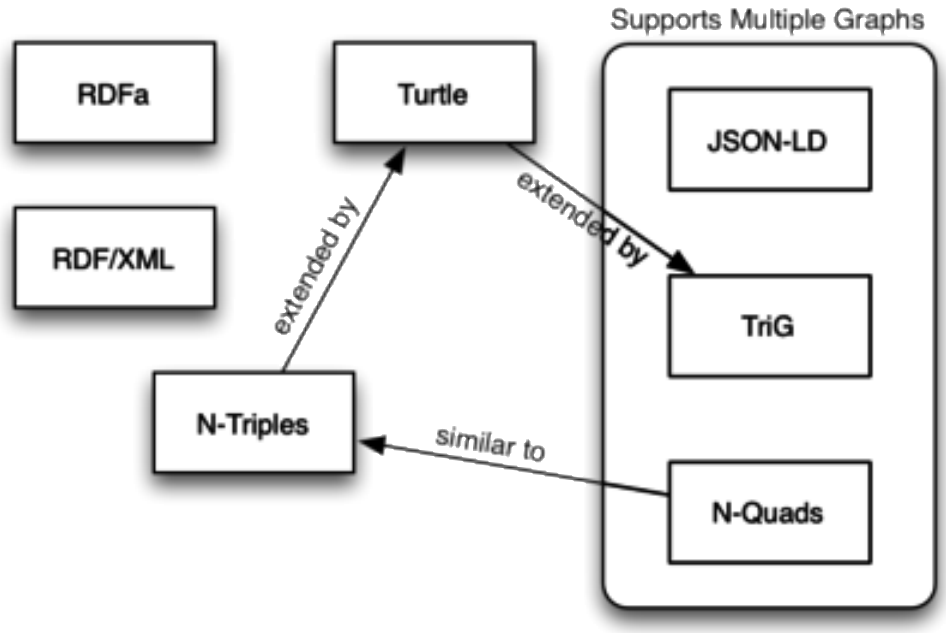


RDF Conventions



Serialization Syntaxes

- RDF has a historical XML syntax and several other syntaxes: Turtle, TriG, JSON-LD, N-Triples, N-Quads



RDF/XML capturing graphs into trees

```
<?xml version="1.0" encoding="utf-8" ?>
<rdf:RDF
                xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
                xmlns:dbo="http://dbpedia.org/ontology/"
                xmlns:foaf="http://xmlns.com/foaf/0.1/" >
  <rdf:Description rdf:about="http://dbpedia.org/resource/George_Clooney">
    <dbo:birthDate  rdf:datatype="http://www.w3.org/2001/XMLSchema#date">1961-05-06</dbo:birthDate>
    <dbo:birthPlace rdf:resource="http://dbpedia.org/resource/Lexington,_Kentucky">
    <foaf:givenName xml:lang="en">George</foaf:givenName>
  </rdf:Description>
</rdf:RDF>
```

N-Triples: a minimalist syntax

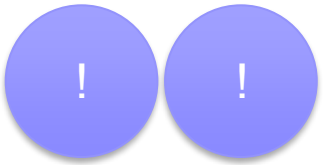
<http://dbpedia.org/resource/George_Clooney>
<http://dbpedia.org/ontology/birthDate>
"1961-5-6"^^<http://www.w3.org/2001/XMLSchema#date> .

<http://dbpedia.org/resource/George_Clooney>
<http://dbpedia.org/ontology/birthPlace>
<http://dbpedia.org/resource/Lexington,_Kentucky> .

<http://dbpedia.org/resource/George_Clooney>
<http://xmlns.com/foaf/0.1/givenName>
"George"@en .

<http://dbpedia.org/resource/George_Clooney>
<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<http://dbpedia.org/ontology/Person> .

Turtle: the most popular syntax




```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix dbr: <http://dbpedia.org/resource/> .

<http://dbpedia.org/resource/George_Clooney>
        dbo:birthPlace <http://dbpedia.org/resource/Lexington,_Kentucky> ;
        dbo:birthDate "1961-05-06"^^xsd:date .

[           dbo:name "Blerina" ;
            dbo:partOf dbr:InsideGroup ]

<http://dbpedia.org/resource/George_Clooney>
        dbo:spuse
            [           dbo:name "Blerina" ;
                        dbo:partOf dbr:InsideGroup ] .
```

[…] → refers to triples that have a blank node as subject

2. SPARQL Recap + Additional Information

Additional / more useful info in slides with marks (i)

Query with SPARQL

SELECT what you want

FROM from where you want

WHERE {as you want}

Turtle syntax with question marks for variables:
?x rdf:type ex:Person

Query with SPARQL

- Triples with common subject:

```
SELECT ?name ?fname
WHERE {?x a ex:Person;
           ex:name ?name ;
           ex:firstname ?fname ;
           ex:author ?y . }
```

```
SELECT ?name ?fname
WHERE{?x rdf:type ex:Person.
       ?x ex:name ?name .
       ?x ex:firstname ?fname .
       ?x ex:author ?y . }
```

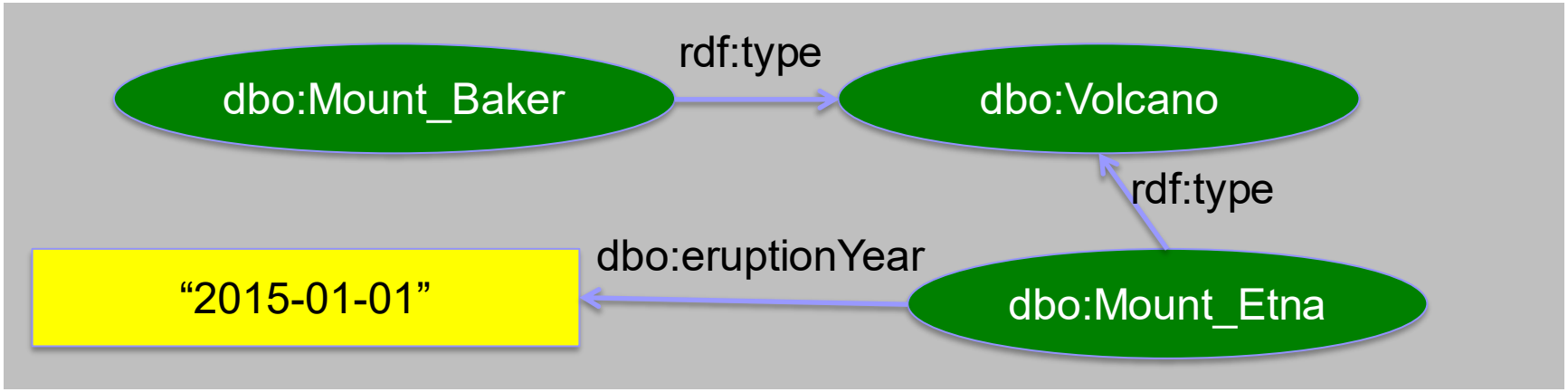
- Several values:

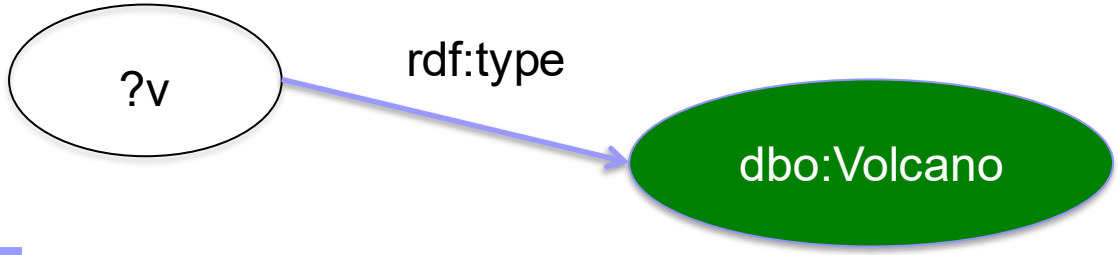
```
?x ex:firstname "Blerina", "Alice" .
```

- Blank nodes as anonymous variables:

```
[ ex:firstname "Blerina" ]
[] ex:firstname "Blerina" .
```

Main Idea of SPARQL Queries





Results:

?v
dbo:Mount_Baker
dbo:Mount_Etna

Components of the SPARQL query

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?v
FROM <http://dbpedia.org>
WHERE {
        ?v rdf:type dbo:Volcano .
}
```

Conventions

Red text means:

"This is a core part of the SPARQL syntax or language."

Green text means:

"This is an example of query-specific text or values that might go into a SPARQL query."

SPARQL lego

Write full URIs:

<http://this.is.a/full/URI/written#out>

Abbreviate URIs with prefixes:

PREFIX ex: <http://this.is.a/URI/prefix#>
ex:university -> <http://this.is.a/URI/prefix#university>

Shortcuts:

a -> rdf:type

Plain literals:

“a plain literal”

Plain literal with language tag:

“buongiorno”@it>

Typed literal:

“23”^^xsd:interger

Shortcuts:

true → “true”^^xsd:boolean
8 -> “8”^^xsd:integer
5.6 -> “5.6”^^xsd:decimal

Variables:

?var1, ?anotherVar, ?and_one_more

Comments:

Comments start with a ‘#’ and
continue to the end of the line

Match an exact RDF triple:

ex:myFiscalCode ex:partNumber “XYZ” .

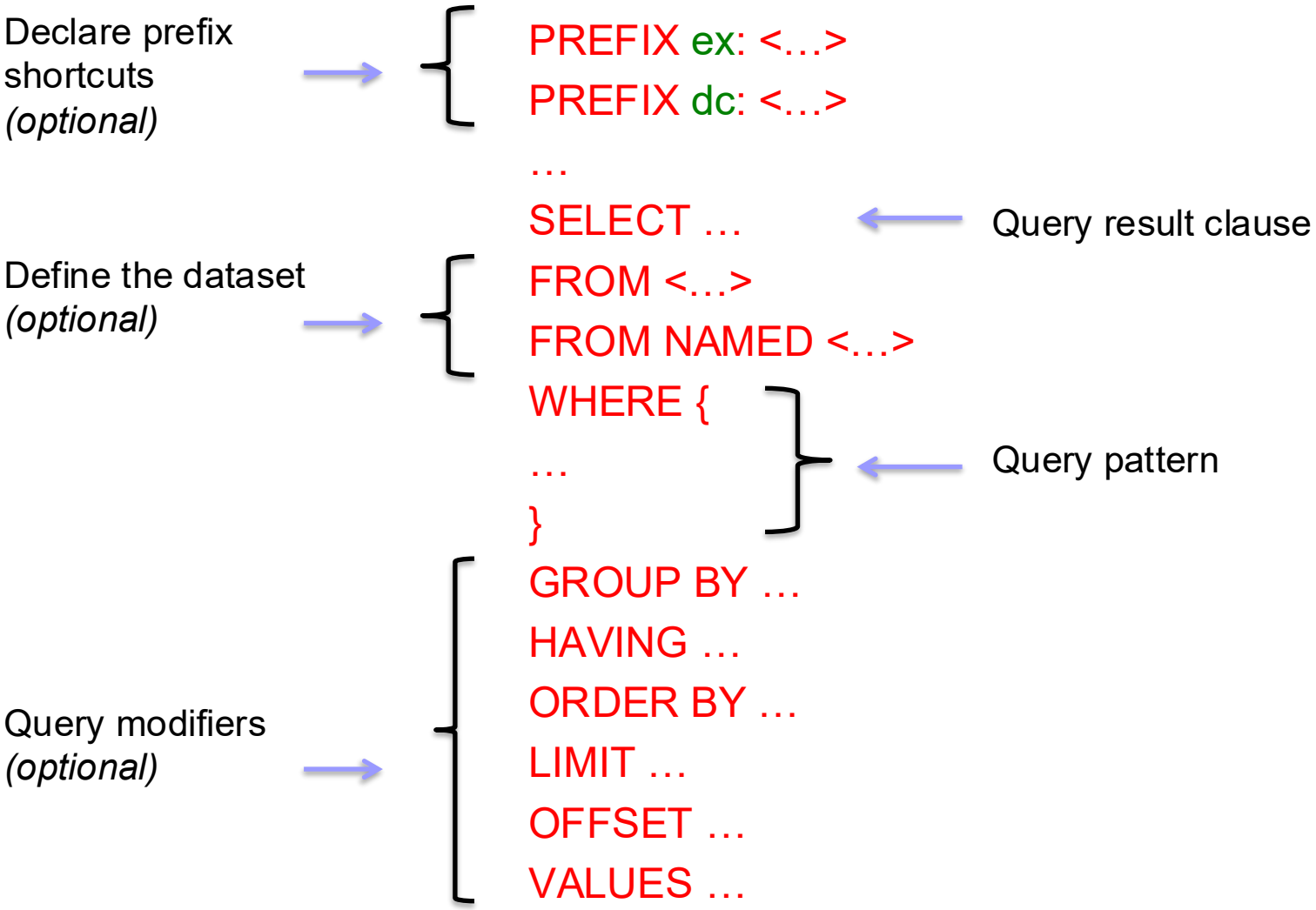
Match one variable:

?person foaf:name “Albert Einstein” .

Match multiple variables:

ex:Unimib ?property ?value .

Anatomy of a SPARQL query



4 Types of SPARQL Queries

SELECT queries

Project out specific variables and expressions:

```
SELECT ?c ?cap (1000 * ?people AS ?pop)
```

Project out all variables:

```
SELECT *
```

Project out distinct combinations only:

```
SELECT DISTINCT ?country
```

Results in a table of values (in XML or JSON)

?c	?cap	?pop
ex:France	ex:Paris	63,500,000
ex:CANADA	ex:Ottawa	32,900,000
ex:Italy	ex:Rome	58,900,000

CONSTRUCT queries

Construct RDF triples / graphs

```
CONSTRUCT {
        ?country a ex:HolidayDestination ;
        ex:arrive_at ?capital ;
        ex:population ?population .
}
```

Results in RDF triples (in any RDF serialization):

```
ex:France a ex:HolidayDestination;
        ex:arrive_at ex:Paris ;
        ex:population 63500000 .
ex:Canada a ex:a ex:HolidayDestination;
        ex:arrive_at ex:Ottawa ;
        ex:population 32900000 .
```

Describe queries

Describe the resources matched by the given variables:

```
DESCRIBE ?country
```

Results are RDF triples (in any RDF serialization):

```
ex:France a geo:Country ;
ex:continuent geo:Europe ;
ex:flag http://_/flag-france.png ;
```

ASK queries

Ask whether or not there are any matches:

```
ASK
```

Results is either "true" or "false" (in XML or JSON):

True, false

USEFUL PATTERNS
EXMPLAINED with

SELECT

Optional Pattern

- When part of graph pattern is not mandatory

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?person ?name
WHERE {
        ?person foaf:homepage <http://blerinaspahiu.com> .
OPTIONAL
        { ?person foaf:name ?name . }
}
```

Alternative Patterns

- Union of results of graph patterns

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?person ?name
WHERE {
        ?person foaf:name ?name .
                {
                 ?person foaf:homepage <http://website.info>.
                }
                UNION
                {
                 ?person foaf:homepage <http://site.info>.
                }
}
```

Difference (~ can handle negation)

- Remove results that match a pattern

```
PREFIX ex: <http://www.example.org#>
SELECT ?x
WHERE {
        ?x a ex:Person
MINUS { ?x a ex:Man }
}
```

Predefined Variable Values

- Results where part of the bindings are predefined

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?person ?name
WHERE {
        ?person foaf:name ?name .
}
VALUES ?name { "Blerina" "Loris" "Alice" }
```

Variable Binding

- Results where part of the bindings are computed

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?person ?name
WHERE {
        ?person ex:fname ?fname ;
                ex:lname ?lname .
BIND (concat(?fname, ?lname) AS ?name)
}
```

Distinct

- Keep one occurrence of similar results with same values for same variables

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT DISTINCT ?name
WHERE {
        ?person foaf:name ?name .
}
```

Filter Results by Values


```
PREFIX ex: <http://www.example.org#>
SELECT ?person ?name
WHERE {
        ?person rdf:type ex:Person ;
                ex:name ?name ;
                ex:age ?age .
FILTER (xsd:integer(?age) >= 18)
}
```

Test Values

- Test and compare constants, variables and expressions
- Comparators: , =, <=, >=, !=
- Regular expressions: regex(?x, "A.*")
- Test variable values: isURI(?x), isBlank(?x), isLiteral(?x), bound(?x)

Strings and Literals

- string inclusion
  CONTAINS(lit1,lit2),STRSTARTS(lit1,lit2),STRENDS(lit1,lit2)
- create literal with datatype
  STRDT(value, type)
- create literal with language
  STRLANG(value, lang)
- concatenate strings
  CONCAT(lit1,…,litn)
- extract substring
  SUBSTR(lit, start [,length])

Boolean Conectors

- And: &&
- Or: ||
- Not: !
- ()

Branching Expression

- Usual test: if then else

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT *
WHERE { ?x foaf:name ?name ;
                    foaf:age ?age .
FILTER ( if (langMatches(lang(?name), "IT"), ?age >=
18, ?age >= 21) )
}
```

Presence / Absence of a Pattern

- exists checks whether a pattern occurs in the graph
- not exists checks whether a pattern does not occur in the graph

```
SELECT ?name
WHERE {
        ?x foaf:name ?name .
FILTER NOT EXISTS { ?x dbo:age -1 }
}
```

Specify Default Graph

```
PREFIX ex: <http://www.example.org#>
SELECT ?student
FROM <http://www.example.org/dataset1.rdf>
FROM <http://www.example.org/dataset2.rdf>
WHERE { ?student ex:registeredAt ?x . }
```

Query Remote SPARQL Endpoint

```
SELECT ?x
WHERE {
   SERVICE <http://dbpedia.org/sparql> {
   ?x rdfs:label "Alice"@it .
   }
}
```

Order and Limit Results

- E.g.: sort results by name from n° 21 to n° 40

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name
WHERE {
        ?x foaf:name ?name .
}
ORDER BY ?name
LIMIT 20
OFFSET 20
```

Aggregate Queries

- Group results by variable(s) values: group by
- Aggregate values: count, sum, min, max, avg, group_concat, sample
- Filter aggregated values: having

```
PREFIX ex: <http://www.example.org#>
SELECT ?student
WHERE { ?student ex:score ?score . }
GROUP BY ?student
HAVING(AVG(?score) >= 10)
```

CONSTRUCTS

CONSTRUCT query

- Result of query is a fresh new RDF graph

```
PREFIX ex: <http://www.example.org#>
PREFIX dbo: <http://dbpedia.org/resource/>
CONSTRUCT { ?student a dbo:Person . }
WHERE { ?student a ex:Student . }
```

DESCRIBE

DESCRIBE query

- Discover unknown data

```
DESCRIBE <http://dbpedia.org/resource/Milan>

PREFIX foaf: <http://xmlns.com/foaf/0.1/>
DESCRIBE ?x WHERE { ?x foaf:name "Alice" }
```

ASK

ASK query

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
ASK { ?person foaf:age 130 . }
```

DATA MANIPULATION

SPARQL Updates

- Load
- Delete
- Insert
- Copy
- Move
- Add

Please refer to https://www.w3.org/TR/rdf-sparql-query/ for SPARQL specification

FOR DETAILS